

The diameter of Hanoi graphs

Daniel Berend^{a,*}, Amir Sapir^{b,*}

^a Departments of Mathematics and Computer Science, Ben-Gurion University, Beer-Sheva 84105, Israel

^b Department of Computer Science, Ben-Gurion University, Beer-Sheva 84105, Israel

Received 22 September 2004; received in revised form 13 September 2005

Available online 20 January 2006

Communicated by M. Yamashita

Abstract

Many questions regarding the Tower of Hanoi problem have been posed and answered during the years. Variants of the classical puzzle, such as allowing more than 3 pegs, and imposing limitations on the possible moves among the pegs, raised the analogous questions for those variants. One such question is: given a variant, and a certain number of disks, find a pair of disk arrangements such that the minimal number of moves required for changing from the first to the second is maximal over all pairs. One of the main results of the paper is identifying these for the Cyclic_h variants—the variants with h pegs arranged along a uni-directional circle—to be the pairs of perfect configurations where the destination peg is right before the source peg.

© 2005 Elsevier B.V. All rights reserved.

Keywords: Algorithms; Graph algorithms; Tower of Hanoi; Directed graphs; Graph diameter

1. Introduction

The Tower of Hanoi puzzle is a well known game, where the player is given 3 pegs and a certain number n of disks of distinct sizes, and is required to transfer them from one peg to another. Initially all disks are placed on the first peg (the source) ordered by size, with the smallest at the top and the largest at the bottom. The goal is to transfer them to the third peg (the destination), using only legal moves, which are the moves obeying the following rules:

- (1) At each step only one disk is moved;
- (2) The moved disk must be a topmost one;

- (3) At any moment, a disk cannot reside on a smaller one (these are the legal disk arrangements).

The puzzle, composed over a hundred years ago by Lucas [13], is a rich source for questions. In some of them, one is clearly led to guess a certain answer, which turns out surprisingly to be incorrect [4]. One such instance is the following:

Question 1. Find a pair of disk arrangements (henceforth *configurations*) requiring the longest sequence of moves in order to get from one to the other.

Intuition directs us to believe that such configurations are *perfect*—configurations in which all disks are stacked on a single peg. However, this is not the case for the general 4-peg problem [10], as will be detailed in Section 1.2.

* Corresponding author.

E-mail addresses: berend@cs.bgu.ac.il (D. Berend), amirsa@cs.bgu.ac.il (A. Sapir).

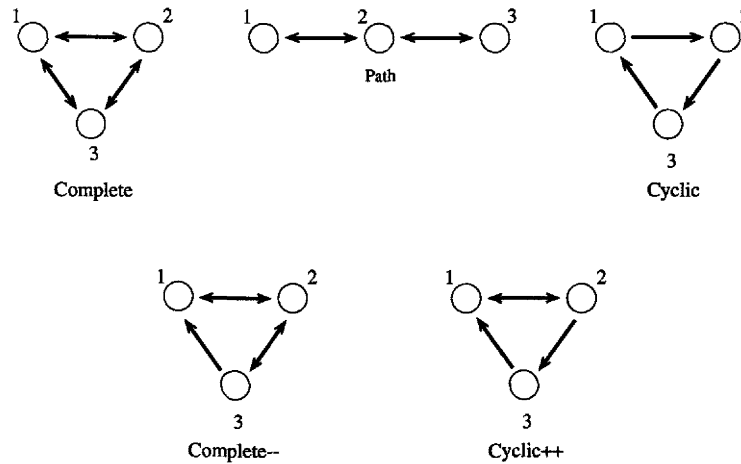


Fig. 1. The 3-peg variants.

1.1. Previous results

Limiting movements between some pairs of pegs, we obtain a rich source for variants, best represented by digraphs—*variant graphs*. Namely, at each vertex of the graph we have a peg, and a disk may be moved from one peg to another only if there exists an arc from the former to the latter. For 3 pegs there are 5 (up to isomorphism) variants, which we name **Complete**(K_3), **Path**, **Cyclic**, **Complete--**, and **Cyclic++**, as seen in Fig. 1. (Throughout this paper we consider only strongly connected graphs, since only on them it is possible to move a tower of any number of disks from any peg to any other peg [12].) In [16,19] **Path** has been discussed, and **Cyclic** was solved in [2]. A single optimal algorithm for all these variants for the tasks of moving from one perfect configuration to another, and explicit formulas for the other two variants, were obtained in [15].

A natural extension of the original problem is obtained by adding pegs. One of the earliest versions is Reve's Puzzle [5, pp. 1–2]. The general setup of the problem, with any number > 3 of pegs and any number of disks, was suggested in [17], with solutions in [18,7]. These and several other algorithms were later shown to be identical [9]. Analysis of the solution reveals, somewhat surprisingly, that the required number of moves grows sub-exponentially, at the rate of $\Theta(\sqrt{n}2^{\sqrt{2n}})$ (cf. [19]). Incorporating movement restrictions with 4 pegs and above, we have a huge number of graphs (83 non-isomorphic strongly connected digraphs on 4 vertices already).

Many other aspects and generalizations, such as in [6,14,1], were studied. We refer to [4,19] for the de-

scription of the history of the problem, and the current state of the field. An extensive bibliography on Tower of Hanoi papers can be found in [20].

1.2. Problem description

The answer to Question 1 for **Complete** has been long known: the transition from one perfect configuration to another requires at least as many moves as are required for a transition between any two configurations [8,21]. However, this is not the case for the general 4-peg problem.

Consider the variant described by K_4 , with 15 disks. The number of moves, starting at a perfect configuration and ending at any other perfect configuration, is 129 (cf. [4]), whereas in [10] there were found some pairs of configurations which are further away. For example, the configuration in which disks 7, 9, 10, 14 are on peg 1, disks 4, 8, 11, 15 are on peg 2, disk 13 is on peg 3, and disks 1, 2, 3, 5, 6, 12 are on peg 4, is 130 moves away from the perfect configuration with all disks on peg 1 [11].

The graphs which are the main object of consideration in this paper are uni-directional cycles, **Cyclic_n**. Namely, there are n pegs arranged in a circle, and we can move a disk residing on some peg only to the next peg in the clockwise direction. It is interesting to examine these graphs since they are among the ‘least connected’—the in-degree and the out-degree of each vertex are both 1. Thus, among all strongly connected digraphs on n vertices, **Cyclic_n** seems a good candidate for being the graph requiring the largest number of moves for transferring a tower of disks. Our main result is that, for any fixed numbers of pegs n and disks m , in

Cyclic_h , the longest task is moving from the perfect configuration on any peg j to that on peg $j - 1$ (modulo h).

The main theorems are presented in Section 2, and their proofs—in Section 3.

The authors express their gratitude to A.M. Hinz, R. Korf, P. Stockmeyer and the referees for their comments on the paper.

2. The main results

First, we introduce some definitions and notations. A problem instance is given by a variant graph and two configurations C_1 and C_2 , and we are required to move from the former to the latter in the minimal number of moves. We denote such a task by $C_1 \rightarrow C_2$, and consider its *length* to be the minimal number of moves required to perform it. Occasionally, we are interested in the *perfect task* of shifting from a perfect configuration $R_{i,n}$ —the configuration where all the n disks reside on peg i —to another perfect configuration $R_{j,n}$. (Depending on the context, we may omit the subscript denoting the number of disks.) This task will be denoted by $R_{i,n} \rightarrow R_{j,n}$; let $d_{ij,n}$ be the length of the task $R_{i,n} \rightarrow R_{j,n}$, and put $d_n = \max_{i,j} d_{ij,n}$.

Given a digraph G and a positive integer n , the corresponding *Hanoi graph* is the directed graph, whose vertices are all configurations of n disks over G , where there exists an arc from a vertex to another if one can pass from the former to the latter by a single disk move. Thus, the problem of finding a pair of (*source*, *destination*) configurations, for which a maximal number of moves is required to change from *source* to *destination*, may be formulated as the problem of finding D_n , the diameter of the Hanoi graph.

As mentioned previously, a perfect task seems to require usually more steps than a “typical” task. For all 3-peg variants, this is indeed the case.

Theorem 1. *For any variant graph on 3 vertices:*

$$D_n = d_n, \quad n \geq 1.$$

It is possible to determine also for which pair(s) (i, j) we have $d_{ij,n} = d_n$. In fact, for **Complete**, all six $d_{ij,n}$ ’s coincide due to symmetry. For the other four graphs, some of the pairs (i, j) are not arcs. We shall refer to such a pair as an *inexistent arc*. Consider *perfect tasks over inexistent arcs*—tasks in which all disks are initially on peg i and are to be moved to peg j , where (i, j) is an inexistent arc. In Lemma 1 it will be shown that all perfect tasks over inexistent arcs are

equally long. The proof of Theorem 1 will imply that d_n is the length of such tasks.

What can be said for general graphs about the relation between D_n and d_n ? Clearly, $d_n \leq D_n$. Theorem 2 and the subsequent corollary show that these two quantities are not too far apart.

Theorem 2. *For any variant graph and any number of disks,*

$$D_n \leq d_n + 2 \sum_{i=1}^{n-1} d_i.$$

Corollary 1.

$$D_n \leq (2n - 1)d_n.$$

Theorem 1 cannot be extended to 4-peg variants in general, as is evident from the anomaly discussed previously. However, our main result states that in the Cyclic_h variant such an anomaly cannot occur. Moreover, we prove the transition between the two farthestmost perfect configurations takes more steps than the transition between any other pair of configurations.

Theorem 3. *For all Cyclic_h graphs where $h \geq 3$, and for $n \geq 1$:*

$$D_n = d_n.$$

Note that an explicit expression for the growth rate of d_n in Cyclic_h for any $h \geq 4$ is currently unknown. However, it has been shown in [3] that d_n grows exponentially fast, with established lower and upper bounds.

3. Proofs

The following notations will be used in the proofs. Let C (perhaps with a sub or superscript) denote any configuration. The peg on which disk l resides in C will be denoted by c_l . $C(n - 1)$ represents the configuration on $n - 1$ disks, obtained from C by disregarding the largest disk. For configurations C_1, C_2, C_3, C_4 , we denote $(C_1 \rightarrow C_2) \leq (C_3 \rightarrow C_4)$ if the task $C_1 \rightarrow C_2$ can be done within at most the same number of moves as the task $C_3 \rightarrow C_4$, and $(C_1 \rightarrow C_2) \sim (C_3 \rightarrow C_4)$ if both $(C_1 \rightarrow C_2) \leq (C_3 \rightarrow C_4)$ and $(C_3 \rightarrow C_4) \leq (C_1 \rightarrow C_2)$.

Lemma 1. *Let G be a strongly connected digraph on 3 vertices. If (i_1, j_1) and (i_2, j_2) are both inexistent arcs, then $(R_{i_1,n} \rightarrow R_{j_1,n}) \sim (R_{i_2,n} \rightarrow R_{j_2,n})$ for every $n \geq 1$.*

```

/* The algorithm shifts from configuration  $C$  to  $C'$  over a graph  $G$  with 3 vertices */
/* by recursively identifying whether the largest disk has to move, and, if so, */
/* whether it can do so by a single step.  $E(G)$  is the set of arcs of  $G$ . */
if  $n \geq 1$  then
  if  $c_n = c'_n$  then
    move3pegs( $C(n-1), C'(n-1), n-1$ )
  else
     $aux \leftarrow$  the third peg number (neither  $c_n$  nor  $c'_n$ )
    if  $(c_n, c'_n) \in E(G)$  then
      move3pegs( $C(n-1), R_{aux}, n-1$ )
      move disk  $n$  from peg  $c_n$  to peg  $c'_n$ 
      move3pegs( $R_{aux}, C'(n-1), n-1$ )
    else
      move3pegs( $C(n-1), R_{c'_n}, n-1$ )
      move disk  $n$  from peg  $c_n$  to peg  $aux$ 
      move3pegs( $R_{c'_n}, R_{c_n}, n-1$ )
      move disk  $n$  from peg  $aux$  to peg  $c'_n$ 
      move3pegs( $R_{c_n}, C'(n-1), n-1$ )
    end if
  end if
end if

```

Algorithm 1. move3pegs(C, C', n).

Proof. Due to symmetry, the lemma is obvious for **Path** and **Cyclic**. It is trivial for **Complete**—, since it has only one inexistent arc. In **Cyclic++** there are two such tasks, which are known to be of the same length [15, p. 22]. $\square$

Theorem 1 can be easily proved for **Complete**, by induction on the number of disks, and we shall deal only with the other four graphs.

Proof of Theorem 1. The direction $d_n \leq D_n$ is obvious, so we will focus on proving that $D_n \leq d_n$. We present an algorithm—Algorithm 1—for the task $C \rightarrow C'$ in any variant graph G on 3 vertices. Algorithm 1 (which is not optimal in general) is a generalization of the algorithm presented in [15] for a perfect task (which is optimal for the restricted problem). We shall henceforth refer to the latter algorithm as *the restricted algorithm*. Then we shall prove that the sequence of moves produced by Algorithm 1 for any task whatsoever is not longer than the one produced by the restricted algorithm, when the task is to move a tower with the same number of disks over an inexistent arc.

Algorithm 1 is recursive, and works as follows. The base case is $n = 0$, in which case there is nothing to be done. For $n \geq 1$, to move from configuration C to C' , we distinguish between three cases:

Case I: Disk n resides on the same peg in both C and C' .

We ignore disk n and reduce the problem to one of moving $n - 1$ disks.

Case II: In C disk n resides on peg i and in C' on peg j , where (i, j) is an arc.

The algorithm will behave ‘greedily’ with respect to disk n , by stacking all smaller disks on the auxiliary peg, so that disk n will make a single move. (The algorithm is not optimal though; see Remark 1 below.) Then, all smaller disks will move from the auxiliary peg to their respective destinations.

Case III: In C disk n resides on peg i and in C' on peg j , where (i, j) is not an arc.

The only path from i to j is through aux , and before each move of the largest disk along that path, all other disks will be stacked on the corresponding remaining peg.

We now show that, for any configurations C and C' on n disks, $(C \rightarrow C') \leq (R_{1,n} \rightarrow R_{3,n})$. We proceed by induction. For $n = 0$, our claim is trivial. Suppose it holds for configurations of $n - 1$ disks for some $n \geq 1$, and let C, C' be configurations on n disks.

There are two possibilities to examine, in accordance with Cases II and III above. (Case I is trivial.) It will be convenient to start with the case in which (c_n, c'_n) is an inexistent arc (Case III). By Lemma 1, we may assume, say, that $c_n = 1, c'_n = 3$. Table 1 compares the sequence of moves produced by Algorithm 1 in this case to the optimal sequence of moves generated by the restricted algorithm for the task $R_{1,n} \rightarrow R_{3,n}$ (which will be de-

Table 1

Comparison of a “hard” task with a perfect task along an inexistent arc

Stage	Task	
	$C \rightarrow C'$, where $c_n = 1$ and $c'_n = 3$	$R_1 \xrightarrow{n} R_3$
a	$C \xrightarrow{n-1} R_3$	$R_1 \xrightarrow{n-1} R_3$
b	$C \xrightarrow{[n]} R_2$	$R_1 \xrightarrow{[n]} R_2$
c	$R_3 \xrightarrow{n-1} R_1$	$R_3 \xrightarrow{n-1} R_1$
d	$R_2 \xrightarrow{[n]} R_3$	$R_2 \xrightarrow{[n]} R_3$
e	$R_1 \xrightarrow{n-1} C'$	$R_1 \xrightarrow{n-1} R_3$

noted here by $R_1 \xrightarrow{n} R_3$). The notation $C \xrightarrow{n-1} C'$ refers to the task of moving the smallest $n - 1$ disks from their locations at C to their locations at C' . Similarly, $C \xrightarrow{[n]} C'$ refers to moving only disk n from its location at C to its location at C' .

In stages (b), (c), and (d), precisely the same sequences of moves are produced for both tasks. Stages (a) and (e) of the task $C \rightarrow C'$ require, by the induction hypothesis, no more steps than the corresponding stages of $R_1 \xrightarrow{n} R_3$. Thus the overall sum for the former task is no longer than that for the latter.

Next we consider the case in which (c_n, c'_n) is an arc (Case II). Table 2 lists the moves performed by Algorithm 1 in this case, versus those done by the restricted algorithm, still for a perfect task over an inexistent arc. Since there are several possibilities, we use the subscript *aux* as used in the algorithm. By the induction hypothesis, stage (a) of $C \rightarrow C'$ is not longer than that of $R_1 \xrightarrow{n} R_3$, stage (c) of the former task is not longer than stage (e) of the latter, and stage (b) takes exactly one step in both. Thus the required inequality holds in this case too, which completes the proof. $\square$

Remark 1. Whenever there is an arc from peg i to peg j , the algorithm chooses to move the largest disk

Table 2

Comparison of an “easy” task with a perfect task over an inexistent arc

Stage	Task	
	$C \rightarrow C'$, where $(c_n, c'_n) \in E(G)$	$R_1 \xrightarrow{n} R_3$
a	$C \xrightarrow{n-1} R_{aux}$	$R_1 \xrightarrow{n-1} R_3$
b	$C \xrightarrow{[n]} R_{c'_n}$	$R_1 \xrightarrow{[n]} R_2$
c	$R_{aux} \xrightarrow{n-1} C'$	$R_3 \xrightarrow{n-1} R_1$
d		$R_2 \xrightarrow{[n]} R_3$
e		$R_1 \xrightarrow{n-1} R_3$

along that arc, without considering other options. As was already pointed out in [4], sequences in which the largest disk moves only once might not be optimal. The example described there is of the following task in **Complete** with $n = 3$: initially disks 1 and 2 are on peg 1, and disk 3 is on peg 2 and the goal is to swap between them. Algorithm 1 requires 7 moves for the task, whereas the optimal sequence is 5 moves long.

Proof of Theorem 2. Let $\text{movePerfect}(i, j, l)$ be an optimal algorithm for moving l disks, initially stacked on peg i , to peg j . Based on movePerfect , we develop the (non-optimal) procedure $\text{moveGeneral}(C, C', n)$, designed to perform any task $C \rightarrow C'$ on n disks (see Algorithm 2).

The procedure performs its task in three major steps:

$$C \xrightarrow{n-1} R_{c_n} \xrightarrow{n} R_{c'_n} \xrightarrow{n-1} C'.$$

At the beginning of the first step, disk 1 is moved to reside on top of disk 2, thus generating a 2-disk tower. Then this tower is moved on top of disk 3, creating a 3-peg tower, and so on until all disks are placed at the (original) location c_n of disk n . The second step is a perfect task—moving a tower of n disks from the peg on which they reside now to the peg on which disk n is to

```

/* The algorithm shifts from configuration C to C' over a graph with any number */
/* of vertices. First it accumulates all the disks at the peg on which the largest */
/* disk resides, then it moves the whole tower to the peg on which the largest disk */
/* is to reside, and eventually unloads the disks to their destinations. */
for l ← 1 to n - 1 do
    movePerfect(c_l, c_{l+1}, l)
end for
movePerfect(c_n, c'_n, n)
for l ← n - 1 to 1 do
    movePerfect(c'_{l+1}, c'_l, l)
end for

```

Algorithm 2. $\text{moveGeneral}(C, C', n)$.

reside in configuration C' . In step 3, disks $n-1$ to 1 are moved to their destined locations similarly (in reverse) to the way they were accumulated during step 1.

Algorithm 2 requires

$$d_n + \sum_{i=1}^{n-1} d_{c_i, c_{i+1}, i} + \sum_{i=1}^{n-1} d_{c'_{i+1}, c'_i, i}$$

disk moves. It follows that, denoting by $D(C, C')$ the distance between C and C' in the configuration graph, we have:

$$\begin{aligned} D(C, C') &\leq d_n + \sum_{i=1}^{n-1} d_{c_i, c_{i+1}, i} + \sum_{i=1}^{n-1} d_{c'_{i+1}, c'_i, i} \\ &\leq d_n + 2 \sum_{i=1}^{n-1} d_i, \end{aligned}$$

which completes the proof. $\square$

Algorithm 2 is not optimal, as the following example demonstrates.

Example 1. Consider the following task over K_4 , with $n = 2$. Let C be the configuration in which disk 1 is on peg 1 and disk 2 on peg 2. Take C' to be the configuration in which disk 1 is on peg 1 and disk 2 is on peg 3. Algorithm 2 requires 5 moves (stacking twice, unnecessarily, disk 1 on top of disk 2) whereas a single move suffices.

Proof of Theorem 3. Let C and C' be any configurations of n disks. By showing that the number of moves required for the task $C \rightarrow C'$ does not exceed the number of moves required for $R_1 \rightarrow R_h$ we prove that $D_n \leq d_n$ (the inequality $d_n \leq D_n$ is clear). Without loss of generality, we assume that, in C' , disk n resides on peg h .

For $1 \leq m \leq n$ and $1 \leq j \leq h$, denote by $C_{(j,m)}$ (and similarly $C'_{(j,m)}$) the configuration in which all disks $\geq m$ are on peg j and all others are at the same locations as in C . Thus, in particular, $C'_{(h,n)} = C'$ and $C'_{(h,1)} = R_h$.

We will start with the following “inequality”:

$$(C \rightarrow C'_{(h,n)}) \leq (C_{(1,n)} \rightarrow C'_{(h,n)}). \quad (1)$$

Then we shall establish the following chain of inequalities:

$$\begin{aligned} (C_{(1,n)} \rightarrow C'_{(h,n)}) &\leq (C_{(1,n)} \rightarrow C'_{(h,n-1)}) \leq \dots \\ &\leq (C_{(1,n)} \rightarrow C'_{(h,1)}). \end{aligned} \quad (2)$$

In other words, we claim that it is not more difficult to switch from C to C' than from $C_{(1,n)}$ to R_h .

Considering an optimal solution for $C_{(1,n)} \rightarrow C'_{(h,n)}$, we obtain from it a not longer solution for $C \rightarrow C'_{(h,n)}$ by omitting the first $c_n - 1$ moves of disk n , which proves the inequality in (1). (We can do this since a disk does not obstruct moves of smaller ones.) Next we drop the last $h - c'_{n-1}$ moves of disk $n-1$ from an optimal solution for $C_{(1,n)} \rightarrow C'_{(h,n-1)}$, obtaining a not longer solution for $C_{(1,n)} \rightarrow C'_{(h,n)}$. In fact, this is possible since, in any solution of $C_{(1,n)} \rightarrow C'_{(h,n-1)}$, at the stage when disk $n-1$ is placed on c'_{n-1} for the last time, disk n is between peg $c'_{n-1} + 1$ and peg h . Hence there will be no need to move disk $n-1$ any more.

In general, for $2 \leq l \leq n$, we drop the last $h - c'_{l-1}$ moves of disk $l-1$ from an optimal solution for $C_{(1,n)} \rightarrow C'_{(h,l-1)}$ to obtain a not longer solution for $C_{(1,n)} \rightarrow C'_{(h,l)}$. The fact that this is possible follows as for disk $n-1$. This establishes (2).

The graph Cyclic_h is isomorphic to the graph obtained from it by reversing the directions of all arcs. Hence for any configurations C_1 and C_2 we have

$$(C_1 \rightarrow C_2) \sim (\tilde{C}_2 \rightarrow \tilde{C}_1),$$

where $\tilde{C}_1$ and $\tilde{C}_2$ are the configurations obtained from C_1 and C_2 , respectively, by moving all disks on each peg j to peg $h+1-j$, $1 \leq j \leq h$. In particular:

$$(C_{(1,n)} \rightarrow C'_{(h,1)}) \sim (C'_{(1,1)} \rightarrow \tilde{C}_{(h,n)}),$$

where in $\tilde{C}_{(h,n)}$ mirroring is performed before placement of disks on a designated peg. By the first part of the proof:

$$(C'_{(1,1)} \rightarrow \tilde{C}_{(h,n)}) \leq (C'_{(1,1)} \rightarrow \tilde{C}_{(h,1)}) = (R_1 \rightarrow R_h). \quad (3)$$

Combining (1), (2) and (3) with the obvious fact that $d_n \leq D_n$, we complete the proof. $\square$

References

- [1] J.-P. Allouche, D. Astoorian, J. Randall, J. Shallit, Morphisms, squarefree strings, and the Tower of Hanoi puzzle, *Amer. Math. Monthly* 101 (1994) 651–658.
- [2] M.D. Atkinson, The cyclic Towers of Hanoi, *Inform. Process. Lett.* 13 (1981) 118–119.
- [3] D. Berend, A. Sapir, The cyclic multi-peg Tower of Hanoi, *Trans. on Algorithms*, 2006, in press.
- [4] J.-P. Bode, A.M. Hinz, Results and open problems on the Tower of Hanoi, *Congr. Numer.* 139 (1999) 113–122.
- [5] H.E. Dudeney, *The Canterbury Puzzles (and Other Curious Problems)*, E.P. Dutton, New York, 1908.
- [6] M.C. Er, The complexity of the generalised Cyclic Towers of Hanoi, *J. Algorithms* 6 (1985) 351–358.
- [7] J.S. Frame, Solution to advanced problem 3918, *Amer. Math. Monthly* 48 (1941) 216–217.

- [8] A.M. Hinz, The Tower of Hanoi, *Enseign. Math.* 35 (1989) 289–321.
- [9] S. Klavžar, U. Milutinović, C. Petr, On the Frame–Stewart algorithm for the multi-peg Tower of Hanoi problem, *Discrete Appl. Math.* 120 (2002) 141–157.
- [10] R.E. Korf, Best-First frontier search with delayed duplicate detection, in: *Proc. of National Conf. on AI (AAAI-04)*, 2004, pp. 650–657.
- [11] R.E. Korf, Private communication, 2004.
- [12] E.L. Leiss, Solving the “Towers of Hanoi” on graphs, *J. Combin. Inform. System Sci.* 8 (1) (1983) 81–89.
- [13] É. Lucas, *Récréations Mathématiques*, vol. III, Gauthier-Villars, Paris, 1893.
- [14] S. Minsker, The Towers of Hanoi rainbow problem: coloring the rings, *J. Algorithms* 10 (1989) 1–19.
- [15] A. Sapir, The Tower of Hanoi with forbidden moves, *Comput. J.* 47 (1) (2004) 20–24.
- [16] R.S. Scorer, P.M. Grundy, C.A.B. Smith, Some binary games, *Math. Gazette* 280 (1944) 96–103.
- [17] B.M. Stewart, Advanced problem 3918, *Amer. Math. Monthly* 46 (1939) 363.
- [18] B.M. Stewart, Solution to advanced problem 3918, *Amer. Math. Monthly* 48 (1941) 217–219.
- [19] P.K. Stockmeyer, Variations on the Four-Post Tower of Hanoi puzzle, *Congr. Numer.* 102 (1994) 3–12.
- [20] P.K. Stockmeyer, Tower of Hanoi bibliography, <http://www.cs.wm.edu/~pkstoc/biblio2.pdf>, 2005.
- [21] D. Wood, Adjudicating a Towers of Hanoi contest, *Internat. J. Comput. Math.* 14 (1983) 199–207.